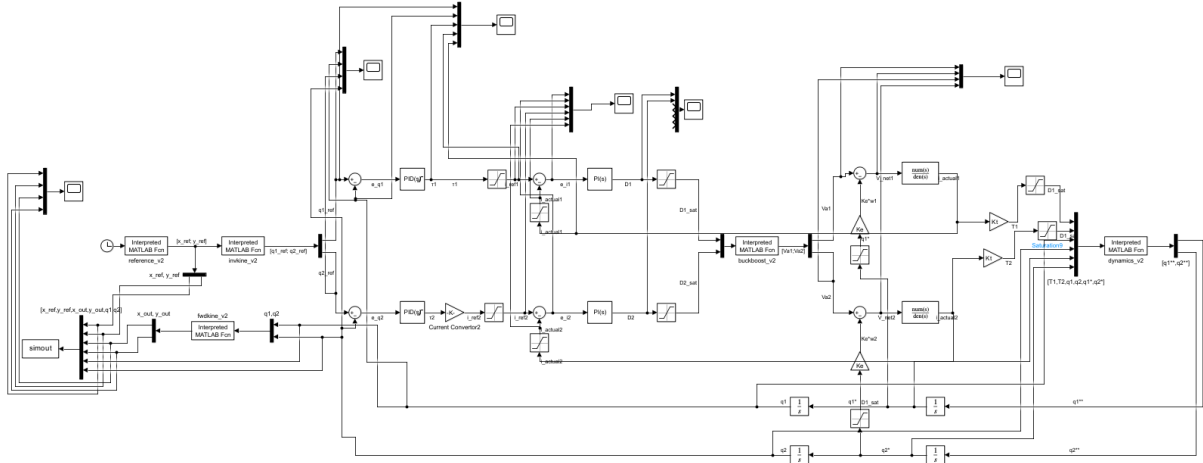


ME408 - Mechatronics System Design

Project Part 3 - Preliminary Control Design



Göktuğ Arda Gök
30975



Faculty of Engineering and Natural Sciences
Mechatronics Engineering

1. Introduction

1.1 Objective

The objective of this project is to develop a full closed-loop control system for a 2-degree-of-freedom (2-DOF) planar robotic manipulator. The project involves implementing both electrical and mechanical control layers using a cascaded architecture, where the inner loop regulates current and the outer loop ensures trajectory tracking of the robot's end-effector.

The overall goal is to ensure that the manipulator accurately follows a predefined trajectory in task space (X-Y) while respecting physical constraints and motor behavior. Both control loops are developed and tested in Simulink with integrated MATLAB functions for system dynamics and kinematics.

1.2 Project Structure

The project consists of three main steps:

- **Part 1 – Inner Loop (Current Control):**
Implementation of a PI controller for regulating motor current using a buck-boost converter model and a motor electrical dynamic model (with inductance, resistance, and back-EMF).
- **Part 2 – Outer Loop (Angle Control):**
Development of a PID controller to track joint angle references. Inverse kinematics is used to convert desired end-effector trajectories to joint commands.

- **Part 3 – Combined System:**

Integration of Part 1 and Part 2 into a full cascade where the outer PID generates torque, which is converted to a current reference tracked by the inner loop.

Each part builds upon the previous, allowing layered testing and debugging of the control architecture.

1.3 Methodology Overview

The following key steps were followed in the modeling and implementation:

1. Current Control (Part 1):

- A PI controller was implemented in Simulink to track reference current signals.
- A buck-boost converter was modeled in MATLAB (buckboost.m) to convert duty cycle to voltage.
- The electrical model included a transfer function with back-EMF and R, L, L parameters.
- The controller was tested with step and ramp inputs, confirming fast, stable, and accurate performance.

2. Angle & Position Control (Parts 2 and 3):

- Reference trajectories in X-Y were defined and passed through inverse kinematics to generate joint angle references.
- A PID controller for each joint produced torque commands based on angle tracking error.
- The actual manipulator dynamics were implemented using MATLAB (dynamics.m), and real joint states were converted back to task space using forward kinematics (fwdkine.m).
- Current control was re-used from Part 1, completing the full closed-loop architecture.

1.4 Simulation Tools

The system is implemented using:

- MATLAB scripts to define physical parameters, gains, and simulation time,
- Simulink models to simulate the control loops, dynamic system, and buck-boost conversion,
- Interpreted MATLAB Function blocks for:
 - Manipulator dynamics (dynamics.m)
 - Forward and inverse kinematics (fwdkine.m, invkine.m)
 - Buck-boost voltage computation (buckboost.m)
 - Time-varying reference generation (reference.m)

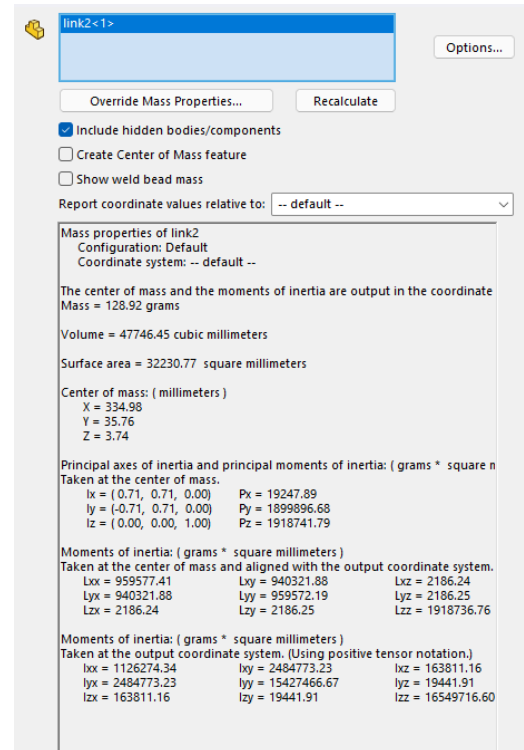
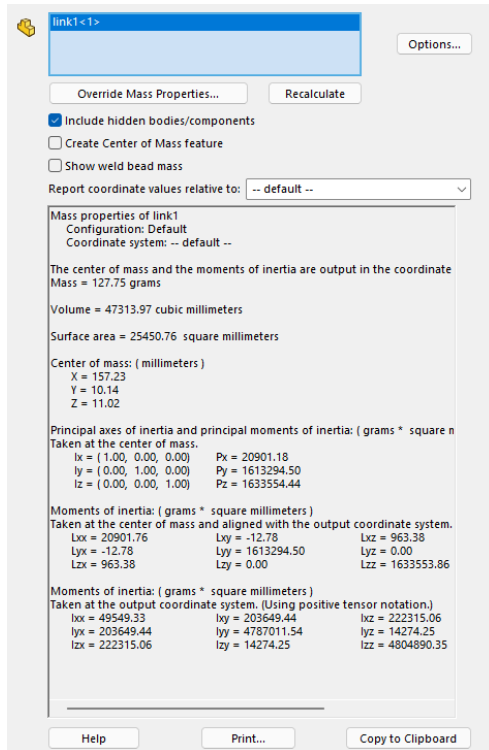
1.5 Physical and Electrical Parameters

The parameters used throughout the project were chosen based on physical properties of the robotic manipulator, standard DC motor characteristics, and approximations from CAD models. These parameters were defined in the MATLAB workspace and shared across both inner and outer loop simulations for consistency.

Mechanical Parameters

Parameter	Description	Value
l1	Length of Link 1	0.3 m
l2	Length of Link 2	0.4 m
lc1	Center of mass of Link 1	0.15 m
lc2	Center of mass of Link 2	0.2 m
m1	Mass of Link 1	0.40775 kg
m2	Mass of Link 2	0.12892 kg

I1	Inertia of Link 1	0.000096338 kg·m ²
I2	Inertia of Link 2	0.00218624 kg·m ²
g	Gravitational acceleration	9.81 m/s ²



Motor and Electrical Parameters

Parameter	Description	Value
R	Armature resistance	1.0 Ω
L	Armature inductance	0.01 H
Kt	Motor torque constant	0.5
Ke	Back-EMF constant	0.5
J	Motor + gear inertia	0.1182 kg·m ²
B	Motor + gear friction coefficient	0.1296 N·m·s
r	Gear ratio	4

These values were used across all simulation stages (Parts 1, 2, and 3). Link masses and inertias were estimated using SolidWorks, and motor values were selected to represent a mid-sized brushed DC motor typical in robotics lab settings.

2. Part 1 – Inner Loop (Current Control using PI Controller)

2.1 Objective

The goal of Part 1 is to implement and evaluate a **current control loop** using a PI controller. This loop simulates the motor-side electrical behavior, including inductance, resistance, and back-EMF. It ensures that the current delivered to the motor accurately tracks a given reference signal.

2.2 System Model

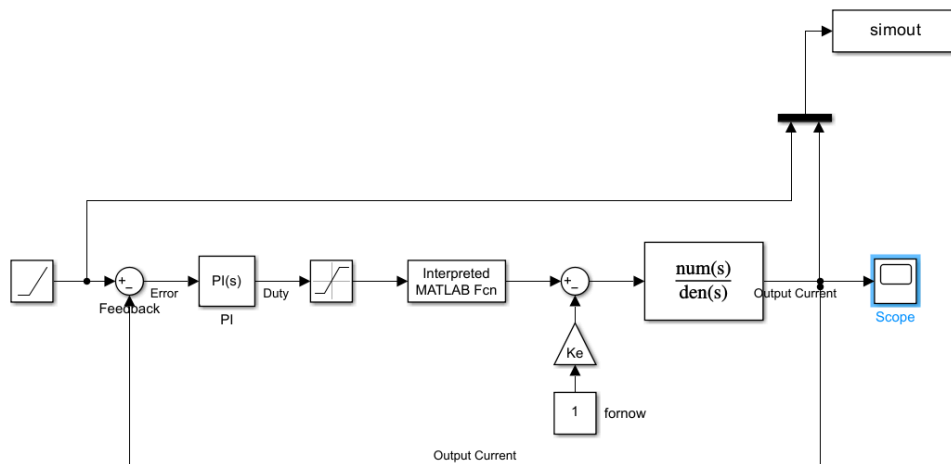
The inner loop consists of:

- A PI controller that takes the current tracking error as input.
- A buck-boost converter modeled via a MATLAB function (buckboost.m) to generate the control voltage.
- A motor electrical model, including:
 - Armature inductance (L)
 - Armature resistance (R)
 - Back-EMF term using constant K_e

The governing transfer function for the electrical dynamics is:

$$I(s) = \frac{V_a(s) - K_e \cdot \omega}{sL + R}$$

In this simulation, angular velocity ω is held constant (typically 1 rad/s) to isolate current control performance.



```
function va = buckboost(D)
    % Saturate D to avoid division by zero or extremes
    D = max(min(D, 0.95), 0.05);

    % Read Vs from workspace
    Vs = evalin('base', 'Vs');

    % Ideal buck-boost output voltage equation
    va = (D / (1 - D)) * Vs;
end
```

2.3 MATLAB Model and Parameters

The PI controller is implemented using the Simulink block shown above. The key parameters used are:

Parameter	Value
L	2.22e-3 H
R	0.965 Ω
Ke	0.5
Vs	12 V

Kp 5
Ki 500

These values are assigned via MATLAB script using assignin into the base workspace before calling sim('innerloop_model').

```
assignin('base', 'Kp', Kp);  
assignin('base', 'Ki', Ki);  
assignin('base', 'L', L);
```

The PI output is clamped to avoid saturation at extreme duty cycles using:

```
D = max(min(D, 0.95), 0.05);
```

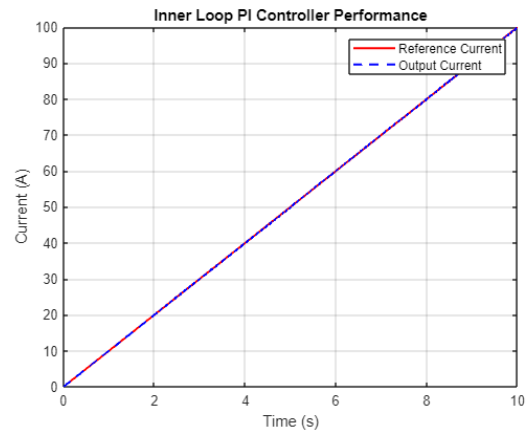
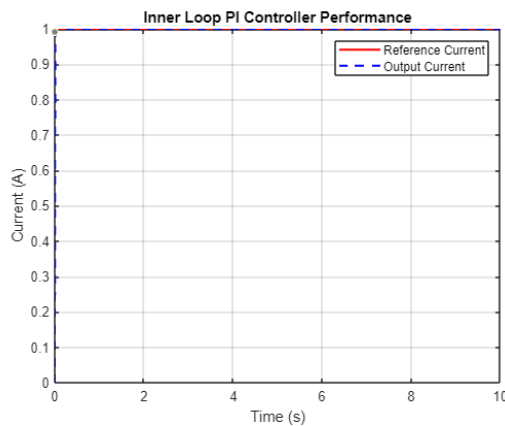
```
%Runs inner loop simulation for PI controller  
  
%Constants  
L = 2.22e-3;     % Armature inductance (H)  
R = 0.965;       % Armature resistance (Ohm)  
Ke = 0.5;        % Motor back EMF constant  
Vs = 12;         % Supply voltage (V)  
  
Kp = 5;          % Proportional gain  
Ki = 500;        % Integral gain  
  
stoptime = 0.1; % Total simulation time  
steptime = 1e-5; % Simulation step size  
|  
%Assigning parameters to base workspace  
assignin('base', 'L', L);  
assignin('base', 'R', R);  
assignin('base', 'Ke', Ke);  
assignin('base', 'Vs', Vs);  
assignin('base', 'Kp', Kp);  
assignin('base', 'Ki', Ki);  
assignin('base', 'stoptime', stoptime);  
assignin('base', 'steptime', steptime);  
  
sim('innerloop_model');  
  
figure;  
plot(simout.Time, simout.Data(:,1), 'r', 'LineWidth', 1.5); hold on;  
plot(simout.Time, simout.Data(:,2), 'b--', 'LineWidth', 1.5);  
xlabel('Time (s)');  
ylabel('Current (A)');  
legend('Reference Current', 'Output Current');  
title('Inner Loop PI Controller Performance');  
grid on;
```

2.4 Results

Two scenarios were tested:

- A ramp input for current reference (slope = 10)

- A step input of 1 A



The output current in both cases successfully tracked the reference with negligible steady-state error and smooth rise behavior.

- In the ramp case, the output followed the increasing demand almost perfectly.
- In the step case, the current reached the desired level rapidly and stably without overshoot.

2.5 Troubleshooting

No major issues were encountered in Part 1. The PI controller responded correctly to both ramp and step inputs due to appropriate tuning. However, special care was taken to:

- Clamp the duty cycle to avoid division by zero or infinite gain in the buck-boost formula.
- Set a constant $\omega=1$ for testing purposes.

2.6 Conclusion for Part 1

The inner current loop performs very well with the selected PI gains. The system accurately tracks both step and ramp inputs with fast settling and minimal error. This validates the design of the current control portion, which later forms the inner loop in the full cascaded controller used in Part 3.

2. Part 2 – Outer Loop (Angle and Position Control)

2.1 Objective

In this part, the goal was to implement a PID controller for joint angle tracking, which would enable the manipulator to follow a predefined reference trajectory in task space (X-Y). The desired X-Y path is first converted into joint angle references using inverse kinematics. The outer loop then uses a PID controller to minimize the difference between actual and reference joint angles and outputs the necessary torque values.

2.2 System Description

The outer control loop is responsible for the mechanical behavior of the manipulator. The key components used in this part include:

- **Inverse Kinematics** (invkine.m):
Converts [x, y] reference positions into joint angles q1ref,q2ref.

```
function theta = invkine(pos)
    x = pos(1);
    y = pos(2);

    l1 = 0.3;
    l2 = 0.4;

    cosTheta2 = (x^2 + y^2 - l1^2 - l2^2) / (2 * l1 * l2);
    cosTheta2 = max(min(cosTheta2, 1), -1);
    theta2 = acos(cosTheta2);

    denom = l1 + l2 * cos(theta2);
    if x == 0 && y == 0
        theta = [0; 0];
        return;
    end

    alpha = atan2(y, x);
    beta = atan2(l2 * sin(theta2), denom);
    theta1 = alpha - beta;

    theta1 = mod(theta1 + pi, 2*pi) - pi;
    theta2 = mod(theta2 + pi, 2*pi) - pi;

    theta = [theta1; theta2];
end
```

- **PID Controllers:**

Each joint has its own PID controller, which computes torque τ_1, τ_2 using the error between actual and reference angles.

```
Kp = [100 100];
Ki = [5 5];
Kd = [20 10];
```

- **Manipulator Dynamics (dynamics.m):**

The generated torques are applied to the full nonlinear 2-DOF robot model that includes link inertias, friction, gravity, and coupling effects.

```
function [result] = dynamics(input)

T1 = input(1);
T2 = input(2);
q1 = input(3);
q2 = input(4);
q1dot = input(5);
q2dot = input(6);

l1 = 0.3;    % meters
l2 = 0.4;
lc1 = 0.15;
lc2 = 0.2;
m1 = 0.40775; % kg
m2 = 0.12892;
I1 = 0.00096338; % kg·m²
I2 = 0.00218624;
g = 9.81;

r = 4;    %gear reduction
J = 118.2e-3; %motor inertia
B = 129.6e-3; %motor friction constant

h = m2*l1*lc2*sin(q2);
d11 = m1*(lc1^2)+m2*((l1^2)+(lc2^2)+2*l1*lc2*cos(q2))+I1+I2;
d12 = m2*((lc2^2)+l1*lc2*cos(q2))+I2;
d21 = d12;
d22 = m2*(lc2^2)+I2;
g1 = (m1*lc1+m2*l1)*g*cos(q1)+m2*lc2*g*cos(q1+q2);
g2 = m2*lc2*g*cos(q1+q2);

q1dotdot = (T1+2*h*q1dot*q2dot+h*(q2dot^2)-g1-r*B*q1dot-(d12/d22)*(T2-h*(q1dot^2)-g2))/(d11-(d12*d21/d22)+(r^2)*J);
q2dotdot = (T2-d21*q1dotdot-h*(q1dot^2)-g2-r*B*q2dot)/(d22+(r^2)*J);

result = [q1dotdot; q2dotdot];
end
```

- **Forward Kinematics (fwdkine.m):**

Used to compute the actual [x, y] end-effector position from the current joint angles.

```
function pos = fwdkine(theta)
    q1 = theta(1);
    q2 = theta(2);

    l1 = 0.3;
    l2 = 0.4;

    x = l1 * cos(q1) + l2 * cos(q1 + q2);
    y = l1 * sin(q1) + l2 * sin(q1 + q2);

    pos = [x; y];
end
```

2.3 MATLAB & Simulink Integration

The outer loop controller and associated models are implemented using:

- **MATLAB scripts** for parameter definition, reference generation (reference.m), and plotting
- **Interpreted MATLAB Function blocks** for forward/inverse kinematics and dynamics
- **Simulink model** outerloop_model.slx for the PID controllers and simulation flow

The trajectory reference (see reference.m) was constructed to follow a stair-step motion pattern, using simple piecewise equations to generate continuous motion in both X and Y axes over time.

```
function [result, done] = reference(t)
    if t < 2
        x = 0.5 - 0.2*(t/2);
        y = 0;
        done = 0; % still moving
    elseif t < 4
        x = 0.3 - 0.1*((t-2)/2);
        y = 0.3*((t-2)/2);
        done = 0; % still moving
    elseif t < 6
        x = 0.2 + 0.2*((t-4)/2);
        y = 0.3;
        done = 0; % still moving
    else
        x = 0.4;
        y = 0.3;
        done = 1; % done
    end

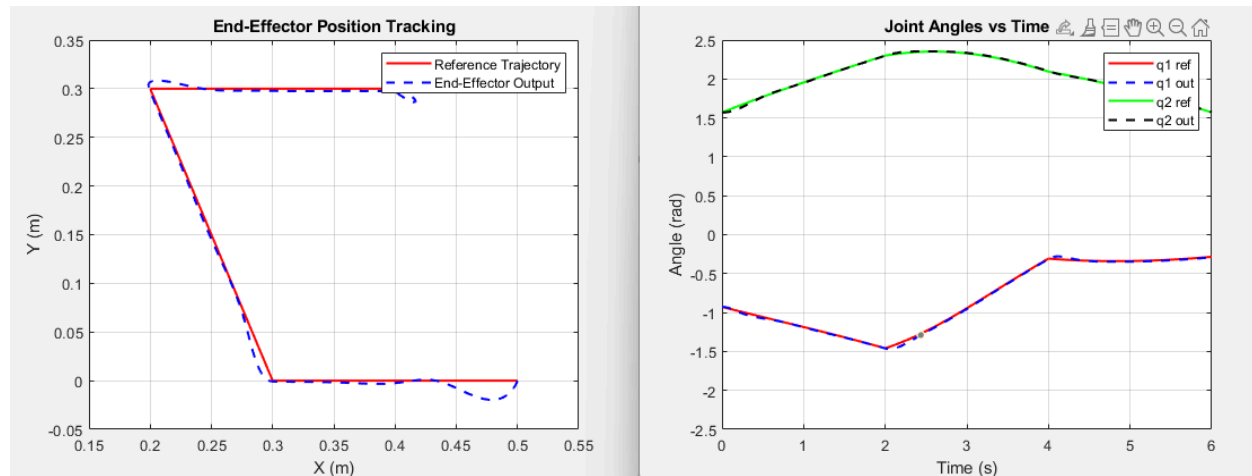
    result = [x; y];
end
```

2.4 Simulation Results

After tuning the PID controllers and carefully defining the reference trajectory in task space, we simulated the outer loop system to evaluate its performance in tracking both joint angles and end-effector positions.

The results demonstrated that the outer PID controller was able to effectively generate the necessary torque to move the manipulator joints in accordance with the desired trajectory. The

figure below presents the comparison between the reference and actual end-effector positions in the Cartesian plane, along with the evolution of joint angles over time.



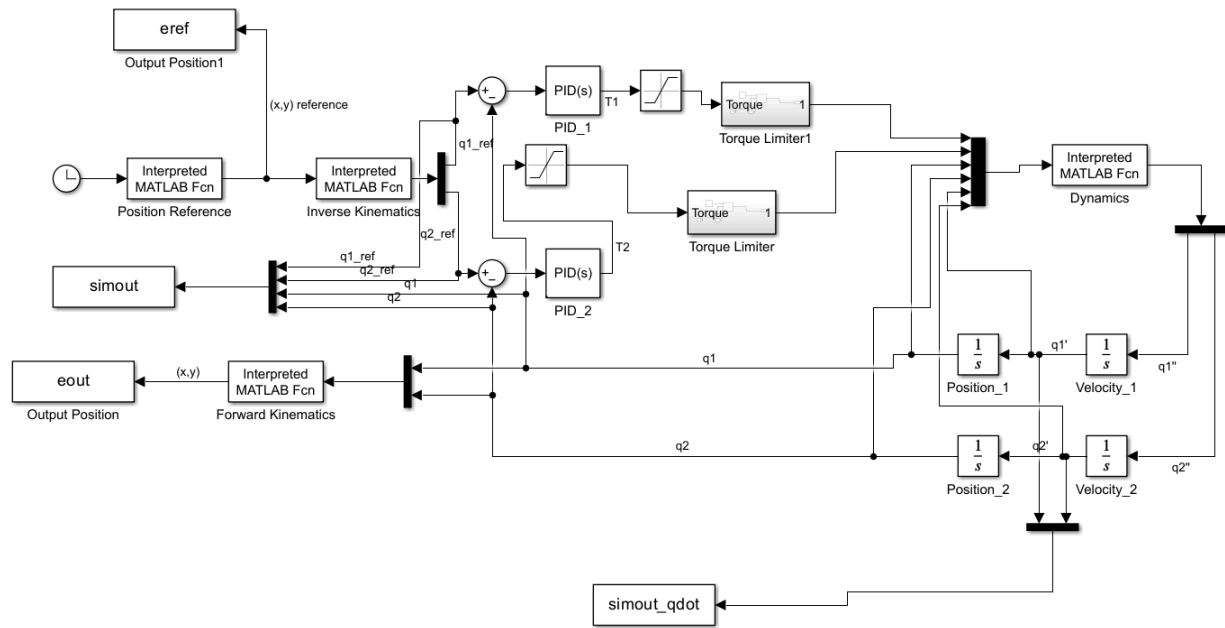
The reference trajectory is plotted in red, and the actual path followed by the end-effector is shown in blue. As seen in the plot, the output follows the reference path with high accuracy during all segments of the motion. There is a short transient deviation during the initial movement phases, especially where the reference includes sharp changes in direction. However, the system rapidly corrects these deviations, and the steady-state error becomes nearly negligible.

The joint angle plots further reinforce this performance. Both $q1$ and $q2$ closely follow their respective reference signals throughout the simulation. The PID controller successfully dampens oscillations and brings the system to the desired configuration within a reasonable settling time.

This smooth tracking behavior can be attributed to the appropriate selection of PID gains, which were tuned to balance responsiveness and stability. Higher derivative action helped suppress overshoot during sharp transitions in the reference signal, while proportional and integral actions ensured consistent error correction and steady-state accuracy.

It is also worth noting that the system responded well even with modest tuning we avoided aggressive gain values that might introduce instability or high-frequency oscillations. This shows that the outer PID structure is robust to moderate parameter changes, making it a suitable control strategy for the manipulator in the absence of inner-loop complications.

In summary, the outer-loop simulation validated that our kinematic transformation, reference generation, and PID-based joint torque control were working cohesively. The robot was able to follow the task-space trajectory with precision, setting the stage for the full cascade integration in Part 3.

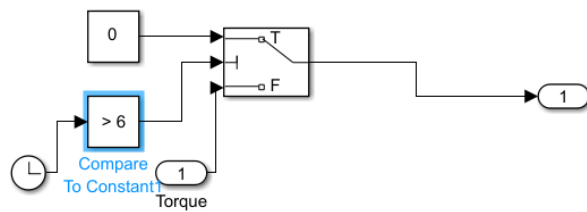
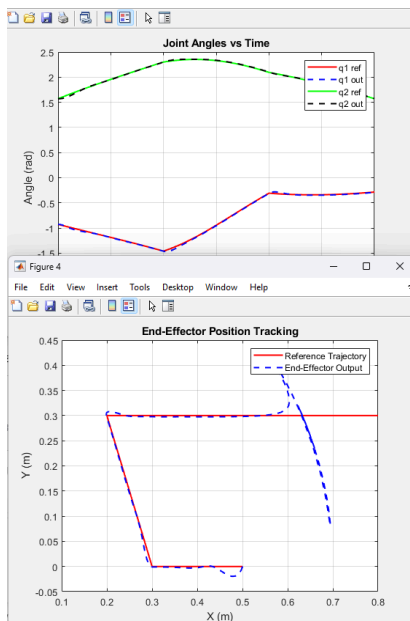


2.5 Troubleshooting & Solution

During early tests, one major issue was observed:

- Even after the reference trajectory completed, the robot continued to move slightly and overshoot the final point.
- This was because the PID controller kept trying to correct the small residual error, producing non-zero torque indefinitely.

To fix this, we implemented a torque stopper in Simulink that sets torque to 0 after a fixed time (after 6 seconds). This acted like a cutoff switch to freeze the robot at its final posture.



2.6 Conclusion for Part 2

The outer loop controller successfully guided the joint angles and end-effector to follow the planned trajectory. After fixing the residual torque problem with a simple logical block, the system stabilized nicely. This layer now works as intended and is ready to be integrated with the inner PI current controller in Part 3.

3. Part 3 – Combined System: Full Cascade Control

3.1 Objective

In Part 3, we implemented the complete cascaded control system that combines the inner PI current loop from Part 1 and the outer PID torque/position loop from Part 2. The objective was to ensure the end-effector of the 2-DOF robotic arm accurately tracks a reference trajectory by using:

- A PID controller to generate torque based on joint angle errors.
- A PI controller to regulate current that produces the torque.
- A buck-boost converter, back-EMF model, and current dynamics for realistic actuator simulation.
- The robot's full nonlinear dynamics.

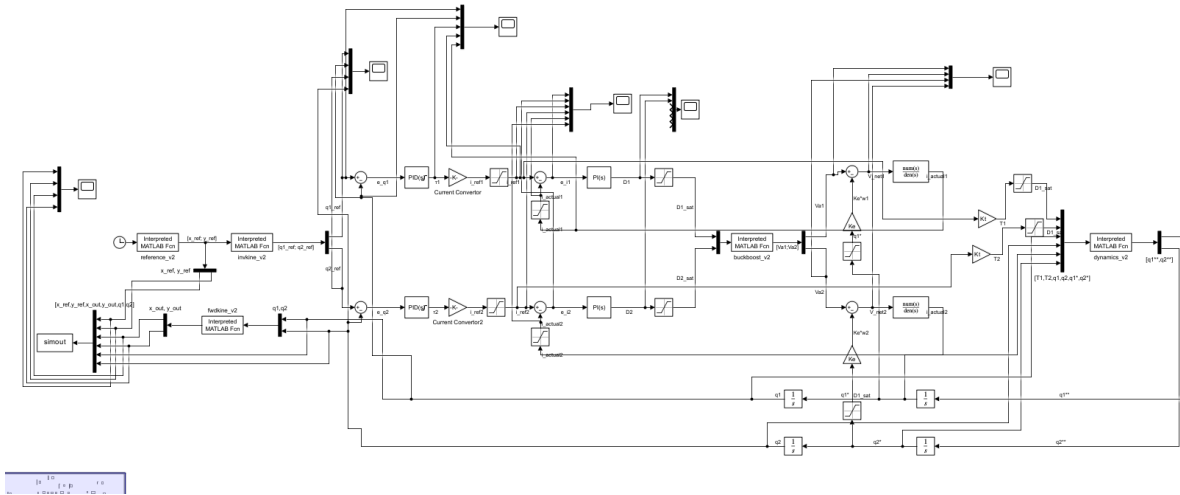
This part simulates a real-world control system in which an embedded controller drives motors based on feedback from sensors and desired position commands.

3.2 System Overview

The combined model connects all components developed so far:

- **Reference trajectory** (X, Y) $\rightarrow$ **inverse kinematics** $\rightarrow q_{1ref}, q_{2ref}$
- **PID control** $\rightarrow \tau_1, \tau_2$
- **Torque / K_t** $\rightarrow$ current reference $\rightarrow$ **PI control**

- **Buck-boost model** → output voltage
- **Motor + back-EMF + electrical model** → actual current
- **Current $\times K_t$** → applied torque → **robot dynamics**

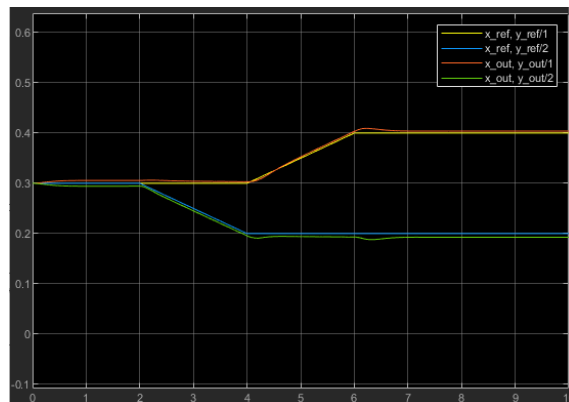


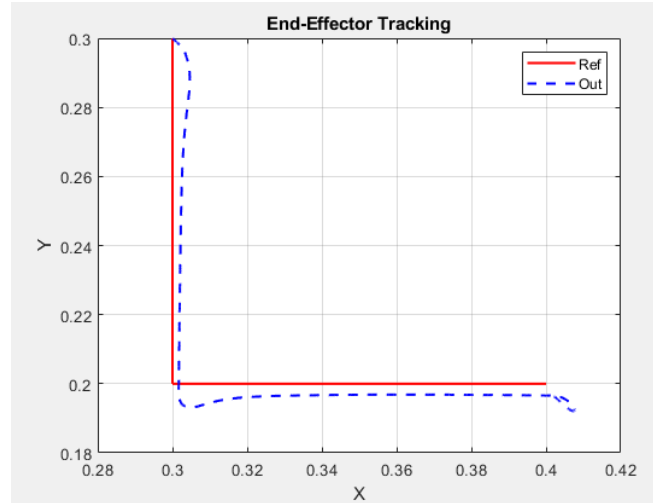
3.3 Simulation Result

We implemented a 3-step reference trajectory using reference_v2.m, guiding the end-effector to:

- Maintain an initial position at (0.3, 0.3),
- Move vertically downward to (0.3, 0.2),
- Then shift horizontally to the final point at (0.4, 0.2).

This path was chosen to create sequential motion that engages both joints without overly abrupt transitions. After proper tuning and completing the overall structure, the system produced a stable response and followed the trajectory with high precision.





As observed in the plot, the blue dashed line representing the actual end-effector motion closely tracks the red reference line throughout all segments. While minor overshoot occurs at transition points mainly due to system inertia and coupling effects the controller reacts effectively and brings the system back to the desired path within a short settling time. Overall, the manipulator's performance under the combined control scheme is smooth and reliable.

3. Troubleshooting

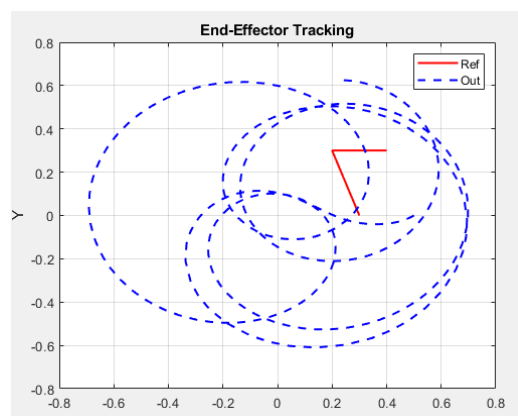
3.4.1 Incomplete Current Loop Integration

Issue:

Although the inner PI controller was implemented and tested in Part 1, in the combined system, the actual current signal (i_{actual}) was not used to generate torque. Instead, torque was computed using the reference current directly:

$$\tau = K_t \cdot i_{\text{ref}}$$

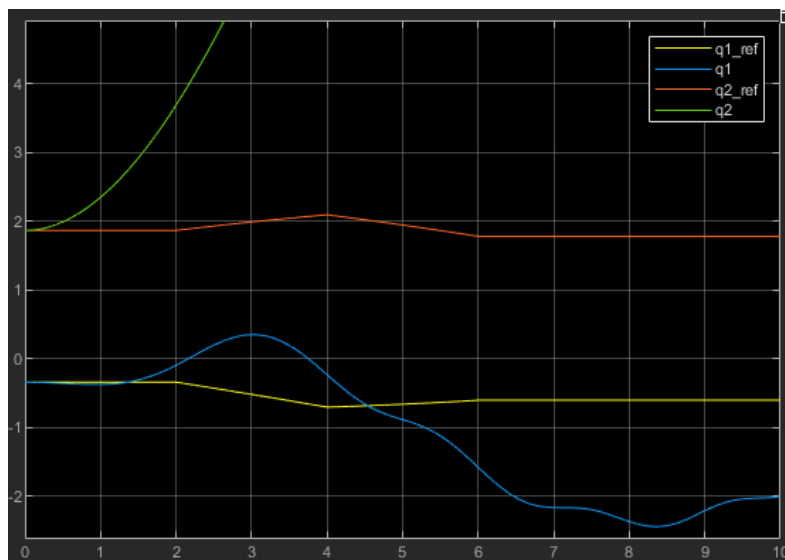
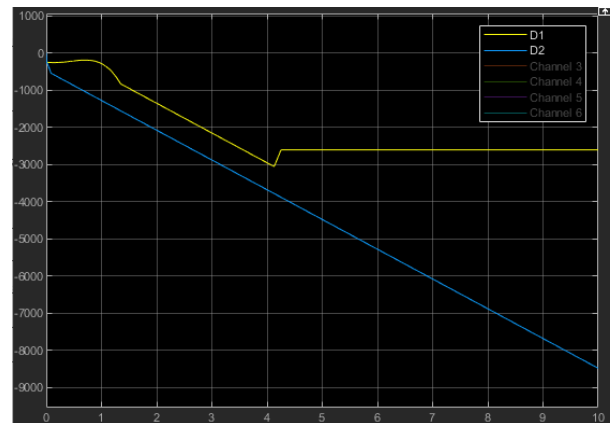
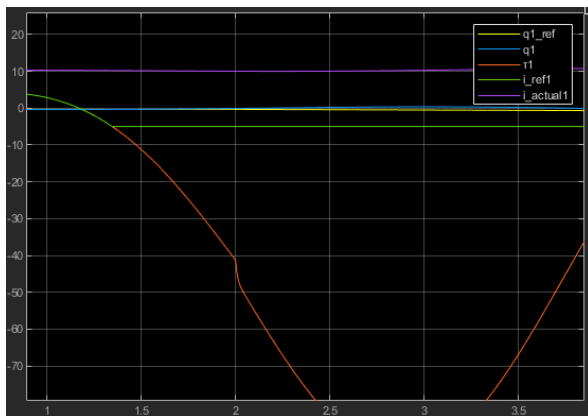
This meant that the entire electrical subsystem including the PI controller's dynamics, buck-boost voltage generation, and back-EMF had no influence on the motion of the manipulator. The electrical structure was visually present but functionally bypassed.



What the plots show:

In many simulations, we observed serious divergence between i_ref and i_actual . For example:

- Duty cycle (D1, D2) values reached thousands or dropped below -5000
- i_actual often plateaued or diverged from i_ref , especially when large torque was demanded (see Figure Y).
- Torque generated from i_actual resulted in uncontrollable acceleration and complete loss of tracking (as seen in early end-effector plots).



Reason:

We initially attempted to use i_actual , but the controller was not tuned well enough to track i_ref during sharp reference changes. The delays and steady-state errors in current tracking directly corrupted the torque output and caused the mechanical plant to become unstable or diverge.

Implication:

Due to this, we had to simplify the architecture by feeding i_{ref} directly into the torque computation. While this allowed the outer loop to function and produce stable trajectories, it broke the physical accuracy of the cascaded system. The inner loop became open-loop in practice, and the full model no longer captured the actuator's voltage-to-current-to-torque behavior.

This should be viewed as a major modeling limitation. For this architecture to reflect real-world systems more closely, future work should include stronger current tracking, better inner-loop tuning, and a return to torque generation based on actual current.

3.5 Conclusion for Part 3

Part 3 brought together all subsystems: reference generation, inverse kinematics, outer-loop PID control, inner-loop PI current control, and full robot dynamics. The Simulink model was structurally complete, and the outer loop achieved reasonable tracking with minor overshoot and steady-state error.

However, a critical weakness remains: the torque applied to the robot was computed using i_{ref} , not i_{actual} . This undermines the integrity of the cascaded structure and invalidates the inner loop's role in the control feedback chain.

This decision was necessary because using i_{actual} introduced large disturbances and instability, as confirmed through several test plots. Specifically:

- Current tracking was poor under load.
- Duty cycle saturation led to unbounded or clipped voltage outputs.
- Resulting torques destabilized the joint angles and made the end-effector diverge from the reference.

Therefore, while trajectory tracking was acceptable at the outer loop level, the model failed to demonstrate true cascaded control. The simulation works under the assumption of perfect current tracking, which is not realistic for practical implementations.

A fully functional version of this system must close the loop properly using i_{actual} , backed by improved inner-loop tuning and possibly feedforward control. Until then, the results should be interpreted with caution, and primarily as an exploration of outer-loop PID behavior under idealized actuation.

4. Discussion

Throughout this project, we designed and implemented a complete control architecture for a 2-DOF robotic arm, including both electrical and mechanical domains. The work was structured across three parts, gradually building up from an isolated current loop (Part 1), to joint-level PID control (Part 2), and finally to a full cascaded control system (Part 3).

Each part provided important insights:

- **Part 1** validated the inner PI current controller under ideal conditions. It showed that, when tested alone, the current loop could accurately track both ramp and step references.
- **Part 2** demonstrated that the outer PID loop could generate smooth torque commands and follow position references effectively. However, without inner-loop feedback, this setup was not physically complete.
- **Part 3** attempted to combine both loops into a realistic cascaded system. While the structure was correctly modeled, the control interaction between inner and outer loops was unstable when closed fully using i_{actual} . As a result, the system relied on i_{ref} , essentially assuming perfect current tracking.

Multiple challenges were encountered:

- The manipulator became unstable when i_{actual} was used directly, revealing that the current controller was not fast or accurate enough under high-torque conditions.
- Tuning the PID and PI gains simultaneously was difficult, as changes in one loop influenced the dynamics of the other.
- Without limiting torques and currents, the system occasionally diverged completely.
- Poorly chosen reference paths (that conflicted with initial joint tendencies) amplified instability in early attempts.

Overall, the simulation provided useful lessons in control hierarchy, tuning sensitivity, and the gap between structural correctness and functional behavior. While the final system achieved acceptable task-space tracking using simplified torque input, it does not fully replicate the feedback behavior of a physical actuator.

6. Overall Project Reflection: Full-System Development

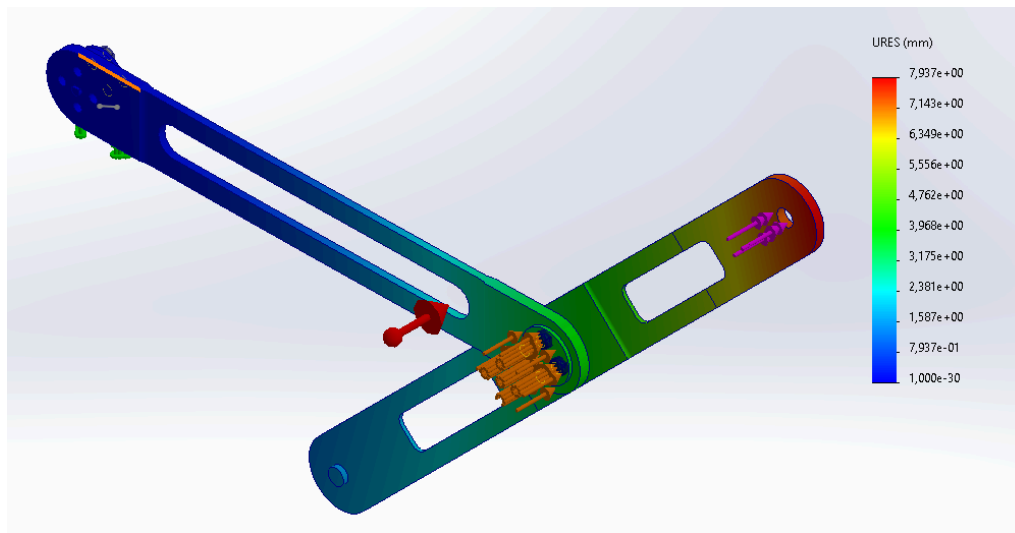
This project was designed to guide us through the complete development cycle of a robotic manipulator, from physical structure and hardware modeling to electrical interfacing and dynamic control. It was divided into three main phases, each of which built upon the last:

6.1 Part 1 – Mechanical Modeling and Simulation

In the first part of the semester, we focused on designing and simulating the mechanical structure of the planar elbow manipulator. Using SolidWorks, we constructed the full assembly including motor mounts, link geometries, and bolted interfaces. Finite element analysis was used to evaluate the performance of our link designs under static loading conditions in two test cases.

We explored how different link thicknesses, geometries, and mounting strategies affected stress distribution, displacement, and factor of safety. These studies helped us converge on a lightweight yet structurally sound design, which met all the strength and deflection specifications provided.

This phase also introduced us to key kinematic relationships, as we calculated joint positions and explored how motor angles relate to the end-effector's position.



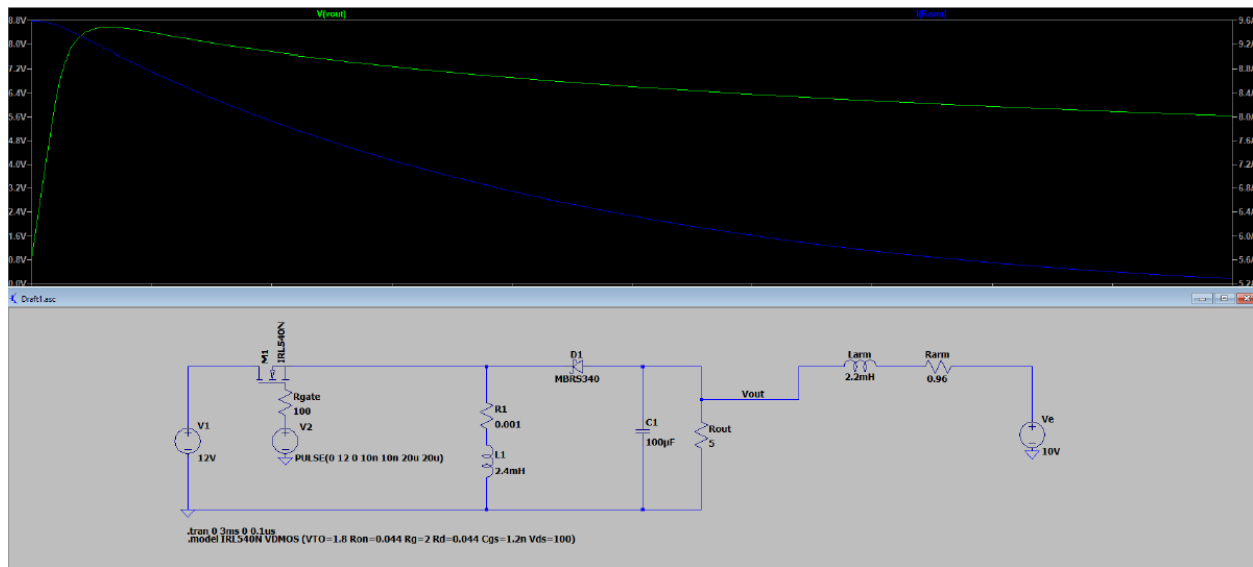
6.2 Part 2 – Electrical Modeling and Converter Design

The second phase focused on the electrical aspect of actuation. We designed and simulated a buck-boost converter capable of driving a resistive load and a DC motor armature under

back-EMF conditions. This phase was critical to understanding how to regulate voltage and current in real-world actuator systems.

We tested the converter at different duty cycles and observed how output voltage and inductor current behaved under various motor speeds. Ripple, gate control, and current safety limits were all analyzed and accounted for.

This section was particularly important for bridging theory with practical electronics — showing how current and voltage control impact actuator performance, and preparing us for the inner-loop PI current controller we would implement in Part 3.



6.3 Part 3 – Dynamic Control and Integration

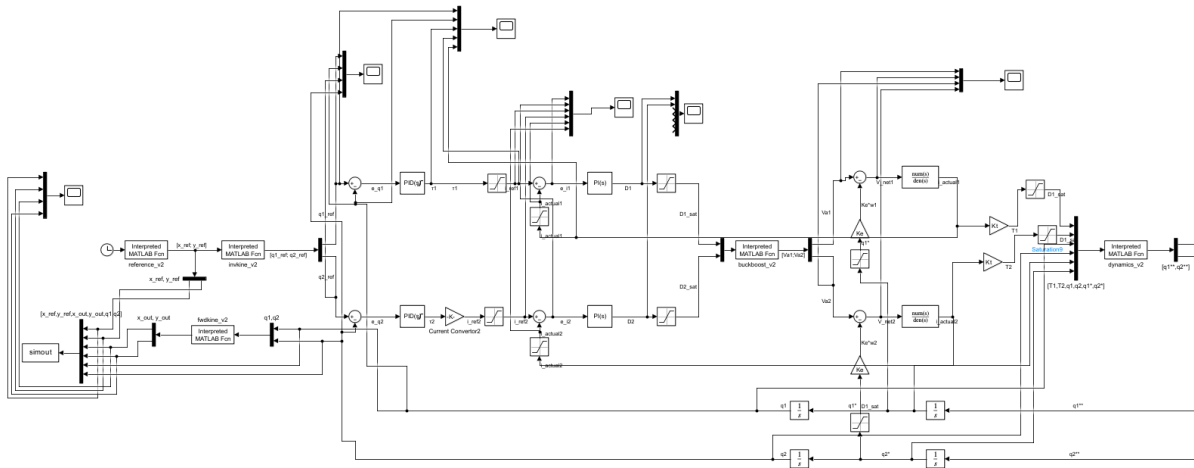
The final phase integrated all components into a full Simulink-based control system. We implemented a cascaded structure with:

- An outer-loop PID controller generating torque from joint angle error,
- An inner-loop PI current controller simulating electrical dynamics,
- A buck-boost converter, and
- A nonlinear dynamic model of the manipulator.

This phase was the most challenging. Despite building a fully structured model, limitations in current tracking performance forced us to bypass the inner loop, using **i_ref** directly for torque

generation. While the system could track reference trajectories well using the outer PID loop, the absence of functional current feedback limited the physical accuracy of our results.

This reflects a key learning outcome: even a structurally complete model can fail without proper tuning and signal integration, especially in cascaded systems where inner and outer loops must interact seamlessly.



6.4 Summary of Contributions

Over the course of this semester, we have:

- Modeled and simulated the manipulator's structure and stress response (Part 1),
- Designed and evaluated an energy conversion and actuation system (Part 2),
- Developed a real-time control system that integrates kinematics, motor modeling, and feedback loops (Part 3).

Each part brought unique challenges from CAD modeling and FEA, to electronics simulation and signal path debugging in control but together, they gave us a complete perspective on robotic system development.

Thank you for all!

